

# Chapter 1: Simple Expressions

We can begin building a programming language with one expression:

```
3+4*5
```

We expect 23. Getting there requires decisions that are easy to overlook when we already know arithmetic. Which characters belong together? Which operation applies to which values? Where does multiplication's higher precedence come from?

The Vertex interpreter in `topic-01-simple-expressions` makes those decisions explicit. It recognizes numbers and the four basic arithmetic operators. Parentheses let us change the grouping. That is enough language to study how source text acquires structure and how that structure determines a result.

This chapter follows `runner.py`, `tokenizer.py`, `parser.py`, and `evaluator.py`. Code excerpts come from those files, with occasional formatting adjustments. Additional examples are identified as such.

## Running an Expression

From the topic directory, run:

```
./vertex '3+4*5'
```

The result is:

```
23
```

Quote the expression so the shell passes it as one argument and does not interpret characters such as `*`. The runner also accepts a filename ending in `.t`:

```
./vertex example.t
```

The supplied file contains the same expression. The runner reads the whole file as one expression, rather than executing a sequence of statements. Whitespace can separate tokens, but a newline does not begin a new statement in this version.

`./vertex` launches the Python runner and passes along its arguments. The runner prints the answer using Python. Vertex does not yet have its own output statement. There are no variables or assignments here either. Those require more language than we have implemented so far.

## Three Stages of Interpretation

The runner connects the stages:

```
tokens = tokenize(expression)
ast = parse(tokens)
result = evaluate(ast)
print(result)
```

The tokenizer turns characters into tokens. The parser organizes those tokens into an **abstract syntax tree (AST)**. The evaluator computes the value represented by that tree.

| Stage     | Input           | Output                     |
|-----------|-----------------|----------------------------|
| Tokenizer | Source text     | List of token dictionaries |
| Parser    | Token list      | Tree of node dictionaries  |
| Evaluator | Expression tree | Numeric value              |

These stages answer different questions. The tokenizer can recognize every character in `3+*4`, but that does not make it a valid expression. The parser rejects the arrangement. Conversely, `3/0` has a valid arrangement but fails when evaluated.

The separation also gives us useful places to inspect a problem. A wrong numeric token calls for a look at tokenization. A wrong grouping calls for a look at parsing. Once the tree is correct, we can examine how evaluation handles its nodes.

## Recognizing Tokens

The tokenizer uses regular expressions, which describe patterns in text. Its patterns begin:

```
patterns = [
    (r"\s+", "whitespace"),
    (r"\d*\.\d+|\d+\.\d*|\d+", "number"),
    (r"\+", "+"),
    (r"\-", "-"),
    (r"/", "/"),
    (r"\*", "*"),
    (r"\(", "("),
    (r"\)", ")"),
    (r".", "error"),
]
```

Each pair associates a pattern with a tag. The `r` prefix makes a Python raw string, which is convenient when writing patterns containing backslashes. The patterns are compiled with `re.compile` before scanning begins.

At the current position, the tokenizer tries each pattern in order:

```
for pattern, tag in patterns:
    match = pattern.match(characters, position)
    if match:
        current_tag = tag
        break
```

match starts at the supplied position. It does not search ahead and skip characters it cannot recognize. The first matching pattern wins. The final error pattern catches an otherwise unrecognized character, such as @.

The number pattern has three alternatives separated by |:

| Alternative           | Examples |
|-----------------------|----------|
| <code>\d*\.\d+</code> | .5, 12.5 |
| <code>\d+\.\d*</code> | 5., 12.5 |
| <code>\d+</code>      | 3, 123   |

Here, `\d` matches a digit, `+` means one or more repetitions, and `*` means zero or more. An escaped dot matches a decimal point. Decimal alternatives come before the integer alternative so that an initial run of digits does not end the number too early.

The minus sign is a separate token. The tokenizer does not treat `-5` as one signed number. Whether a minus token can appear before an expression is a question for the grammar.

## Token Values and Positions

For non-whitespace input, the tokenizer constructs a dictionary:

```
token = {"tag": current_tag, "line": line, "column": column}
if current_tag == "number":
    if "." in value:
        token["value"] = float(value)
    else:
        token["value"] = int(value)
tokens.append(token)
```

A numeric token carries a Python number rather than just the original text. For example, `"3"` becomes the integer 3, while `"3.0"` becomes the floating-point value 3.0. Operator tokens carry their tag and source location but do not need a numeric value.

For the source `3+4*5`, tokenization produces:

```
[
  {"tag": "number", "line": 1, "column": 1, "value": 3},
  {"tag": "+", "line": 1, "column": 2},
  {"tag": "number", "line": 1, "column": 3, "value": 4},
  {"tag": "*", "line": 1, "column": 4},
  {"tag": "number", "line": 1, "column": 5, "value": 5},
  {"tag": None, "line": 1, "column": 6}
]
```

Whitespace is consumed without becoming a token. The tokenizer still updates the line and column as it advances, so later tokens keep their locations in the original input. Lines and columns start at one.

The final token has a tag of `None`. It marks the end of input. That gives the parser something explicit to inspect when no more ordinary tokens remain. It is a token dictionary whose tag is `None`, rather than a bare `None` in the list.

## Describing the Grammar

The parser file gives the grammar in Extended Backus-Naur Form (EBNF):

```
expression = term { ("+" | "-") term }
term = factor { ("*" | "/" ) factor }
factor = <number> | "(" expression ")"
```

An expression starts with a term and may continue with additions or subtractions. A term starts with a factor and may continue with multiplications or divisions. A factor is either a number or a parenthesized expression.

Braces mean that the enclosed part can repeat zero or more times. The vertical bar separates alternatives. Quoted symbols are actual tokens in the language; the unquoted grammar names describe larger pieces.

A single number is therefore a factor, a term, and an expression. We do not need an operator for a valid expression. On the other hand, `-5` does not match the factor rule. Subtraction works in `0-5`, but unary negation is not implemented yet.

The grammar also explains precedence. An expression must obtain complete terms before combining them with `+` or `-`. A multiplication inside a term is grouped before the surrounding addition. This decision belongs to the parser, not to a later search for the most important operator.

## Building an Expression Tree

Our AST uses a number node for a literal:

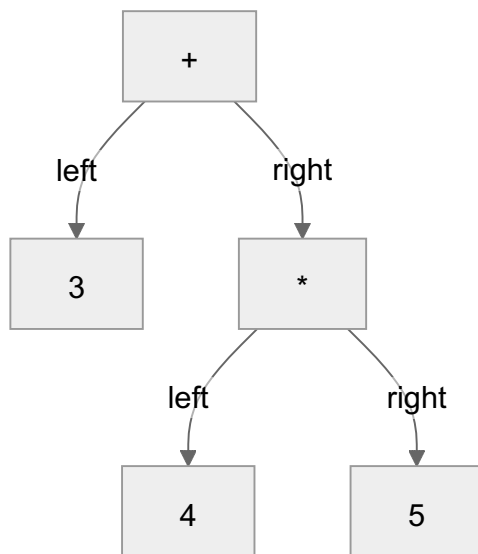
```
{"tag": "number", "value": 3}
```

A binary operator node has an operator tag and two children:

```
{
  "tag": "+",
  "left": {"tag": "number", "value": 3},
  "right": {"tag": "number", "value": 4}
}
```

The children are themselves nodes. Nesting them represents larger expressions without having to put the source text back together.

### The Tree for 3+4\*5



The multiplication belongs to the right child of the addition. Its result supplies one operand of +.

The tree is abstract because it keeps the structure needed for computation without retaining every detail of the source. There are no whitespace nodes. Parentheses influence the tree but do not survive as separate nodes in this implementation. The numeric AST nodes also omit the source positions that were present in the tokens.

### Parsing a Factor

A recursive-descent parser uses functions corresponding to grammar rules. Here, `parse_expression` calls `parse_term`, which calls `parse_factor`. Parentheses allow `parse_factor` to call `parse_expression` again.

Each parsing function returns two things: the node it built and the tokens it has not consumed. The number case in `parse_factor` is:

```
token = tokens[0]
if token["tag"] == "number":
    node = {"tag": "number", "value": token["value"]}
    return node, tokens[1:]
```

Reading  $3+4$  as a factor consumes the 3 and leaves the +, the 4, and the end token. Returning the unused tokens lets the calling function continue at the correct place. These slices create new lists; the parser is not removing elements from the original list in place.

The parenthesis case does more work:

```
if token["tag"] == "(":
    node, tokens = parse_expression(tokens[1:])
    if tokens[0]["tag"] != ")":
        raise SyntaxError(f"Expected ')', got {tokens[0]}")
    return node, tokens[1:]
```

It skips the opening parenthesis and parses a complete expression inside it. Then it requires and consumes the closing parenthesis. The returned node is the inner expression's node. For  $(3+4)*5$ , that addition becomes the left operand of multiplication.

If the next token is neither a number nor an opening parenthesis, the factor function raises a syntax error. This catches an operator where an operand was required, including the second operator in  $3+*4$ .

## Parsing Terms and Expressions

The term parser collects multiplication and division:

```
def parse_term(tokens):
    left, tokens = parse_factor(tokens)
    while tokens[0]["tag"] in ["*", "/"]:
        op = tokens[0]["tag"]
        right, tokens = parse_factor(tokens[1:])
        left = {"tag": op, "left": left, "right": right}
    return left, tokens
```

First it gets a factor. As long as the next token is  $*$  or  $/$ , it consumes that operator and another factor, then creates a new operator node. The previous tree becomes the new node's left child.

This produces **left associativity**. In  $8/4/2$ , the first node represents  $8/4$ . The next node represents  $(8/4)/2$ . Its result is  $1.0$ , whereas  $8/(4/2)$  produces  $4.0$ .

The expression parser has the same structure at the next precedence level:

```
def parse_expression(tokens):
    left, tokens = parse_term(tokens)
    while tokens[0]["tag"] in ["+", "-"]:
        op = tokens[0]["tag"]
        right, tokens = parse_term(tokens[1:])
        left = {"tag": op, "left": left, "right": right}
    return left, tokens
```

For  $3+4*5$ , the first term is the number 3. After reading +, the expression parser asks for another whole term. That term parser consumes  $4*5$  and returns its multiplication node. Only then does the expression parser construct the addition node.

Precedence and associativity are separate choices. Precedence decides how different operator levels group together. Associativity decides how a chain at one level groups. The function calls establish the levels; the loops build left-associated chains.

## Requiring the Whole Input

The public parse function checks that parsing consumed the entire expression:

```
def parse(tokens):
    ast, tokens = parse_expression(tokens)
    if tokens[0]["tag"] is not None:
        raise SyntaxError(f"Unexpected token: {tokens[0]}")
    return ast
```

Without this check, `3 4` could be accepted as the expression `3`, with the second number silently ignored. A helper successfully parsing a prefix is not the same as the full input being valid.

Parentheses make that distinction necessary. While parsing the contents of `(3+4)`, the expression helper needs to stop before `)`, leaving it for the factor function. At the outermost level, `parse` instead requires the end marker.

## Evaluating the Tree

The evaluator follows the AST recursively:

```
def evaluate(ast):
    if ast["tag"] == "number":
        return ast["value"]
    elif ast["tag"] == "+":
        return evaluate(ast["left"]) + evaluate(ast["right"])
    elif ast["tag"] == "-":
        return evaluate(ast["left"]) - evaluate(ast["right"])
    elif ast["tag"] == "*":
        return evaluate(ast["left"]) * evaluate(ast["right"])
    elif ast["tag"] == "/":
        return evaluate(ast["left"]) / evaluate(ast["right"])
    else:
        raise ValueError(f"Unknown AST node: {ast}")
```

A number node is the base case. An operator node evaluates its children and applies the corresponding Python operation to their values. For our example, the left child supplies `3`; the right subtree supplies `20`; the addition returns `23`.

The evaluator does not need to know that `*` has higher precedence than `+`. The parser has already encoded the grouping in the tree. Nor does the evaluator need to find parentheses in the source.

This implementation uses Python's numeric behavior. `/` performs true division, so `6/2` produces `3.0`. Decimal literals become binary floating-point values and can have rounding effects. Division by zero raises an exception. These are semantic choices inherited from the implementation, even though the input language has its own syntax.

## Testing the Implementation

We do not yet have enough of Vertex to write a test suite in Vertex. There is no output statement or assertion facility. For now, we will be content with the unit tests in the Python code.

Run the existing Python unit tests from the topic directory:

```
python3 tokenizer.py
python3 parser.py
python3 evaluator.py
```

Each file calls its test functions when run directly. Python assertions compare actual and expected results. An assertion failure interrupts execution; successful runs reach `done`. These assertions belong to the implementation language, not to the expression language we are building.

The tokenizer tests cover numeric forms and operators, including whitespace handling and an unexpected character. Parser tests check node dictionaries and the unused tokens returned by helpers. Evaluator tests use both manually constructed trees and a tokenized, parsed expression.

Checking the tree matters even when a result looks right. Addition can conceal grouping errors:  $(1+2)+3$  and  $1+(2+3)$  both yield 6. Subtraction and division make associativity mistakes easier to detect.

Additional cases worth checking are:

| Input     | Expected behavior          | Stage to inspect            |
|-----------|----------------------------|-----------------------------|
| $3+4*5$   | 23                         | Precedence in parsing       |
| $(3+4)*5$ | 35                         | Parenthesized factor        |
| $8/4/2$   | 1.0                        | Left associativity          |
| $.5+.5$   | 5.5                        | Decimal tokens              |
| $0-5$     | -5                         | Binary subtraction          |
| -5        | Syntax error               | Missing unary rule          |
| 3 4       | Syntax error               | Complete-input check        |
| $3+*4$    | Syntax error               | Missing operand             |
| $(3+4$    | Syntax error               | Missing closing parenthesis |
| $3@4$     | Unexpected-character error | Tokenization                |
| $3/0$     | Division-by-zero error     | Evaluation                  |

An error is not always evidence that the interpreter is broken. Rejecting an invalid program is part of its job. A useful test checks that rejection happens for the expected reason.



## Practice

Work in a copy of the topic directory.

1. Run `example.t`, then change a copy to  $(3+4)*5$ . Trace why the result changes even though the same numbers and arithmetic operators appear.
2. Print the tokens for  $12 + .5$ . Identify the positions, numeric types, and final marker.
3. Use Python's `pprint` to inspect the tree for  $3+4*5$ . Draw the tree for  $(3+4)*5$  and compare their roots.
4. Call `parse_factor` on the tokens for  $3+4$ . Inspect both return values. Repeat with `parse_term` on  $3*4+5$ .
5. Add Python unit tests for  $8-3-2$  and  $8/4/2$ . Check the tree structure as well as the result.
6. Add Python tests that expect errors for  $3\ 4$ ,  $(3+4$ , and  $3+*4$ . Keep a test for division by zero separate from these syntax cases.
7. Construct a multiplication node manually and pass it directly to `evaluate`. Explain what this tests without involving the tokenizer or parser.
8. As an extension, propose a factor rule for unary negation. Decide what node it would create before changing the tokenizer, parser, or evaluator. Test the extension in your copy with  $-5$ ,  $3*-2$ , and  $-(3+4)$ .

## Questions for Thought and Study

1. Why do we need a parser after recognizing valid tokens?
2. How does a numeric token differ from a numeric AST node in this implementation?
3. Why does the tokenizer try the decimal alternatives before the integer alternative?
4. What does each parsing helper return besides a node?
5. Where does multiplication's higher precedence appear in the code?
6. Why does the term parser group  $8/4/2$  to the left?
7. Why can parentheses disappear from the AST without losing their effect?
8. What would go wrong if `parse` did not inspect the remaining tokens?
9. Which stage rejects  $-5$ , and why does  $0-5$  work?
10. Why can we test this interpreter with Python assertions before our own language supports assertions?

## Further Reading

- **Python regular expressions:** Pattern syntax and matching operations used by the tokenizer.  
<https://docs.python.org/3/library/re.html>
- **Recursive-descent parsing:** Background on parsers organized around grammar rules.  
[https://en.wikipedia.org/wiki/Recursive\\_descent\\_parser](https://en.wikipedia.org/wiki/Recursive_descent_parser)
- **Abstract syntax trees:** The distinction between source notation and the structure used for computation.  
[https://en.wikipedia.org/wiki/Abstract\\_syntax\\_tree](https://en.wikipedia.org/wiki/Abstract_syntax_tree)